

Secure Coding Review Report

Title: Secure Coding Review of a Python Flask Login Application

Objective

The purpose of this task is to perform a secure coding review on a Python-based login application to identify security vulnerabilities, analyze risks, and suggest remediation techniques.

Application Details

- Application Type: Flask Login System
- Programming Language: Python
- Framework: Flask
- Database: SQLite

Scope of Review

The review focuses on:

- Authentication mechanism
- Database queries
- Password handling
- Input validation
- Secure coding practices

Tools and Methods Used

Tools Used

Tool	Purpose
Bandit	Static security analysis for Python
Manual Inspection	Detect logical and coding vulnerabilities
VS Code	Source code review

Application Source Code

Vulnerable Login Code

```
from flask import Flask, request
import sqlite3

app = Flask(__name__)

@app.route('/login', methods=['POST'])
def login():
    username = request.form['username']
    password = request.form['password']

    conn = sqlite3.connect('users.db')
    cursor = conn.cursor()

    query = "SELECT * FROM users WHERE username='" + username + "' AND password='" + password + "'"

    cursor.execute(query)
    result = cursor.fetchone()

    if result:
        return "Login Successful"
    else:
        return "Invalid Credentials"
```

Static Analysis Using Bandit

Installing Bandit

```
pip install bandit
```

Running Bandit

```
bandit -r project_folder/
```

Practical Implementation Steps

Step 1: Install Python

1. Download Python from the official website.
2. Install Python and enable the “Add Python to PATH” option.
3. Verify installation using:

```
python -version
```

Step 2: Install Flask

```
pip install flask
```

Step 3: Install Bandit

```
pip install bandit
```

Step 4: Create Python Project Folder

Create a folder named:

```
secure_login_project
```

Save the vulnerable login code inside `app.py`.

Step 5: Run Application

```
python app.py
```

Step 6: Run Security Scan

```
bandit -r .
```

Screenshot 1 – Installing flask & bandit

```
PS C:\Users\User\OneDrive\Desktop\Task3> pip install flask
Defaulting to user installation because normal site-packages is not writeable
Collecting flask
  Downloading flask-3.1.3-py3-none-any.whl.metadata (3.2 kB)
Collecting blinker>=1.9.0 (from flask)
  Downloading blinker-1.9.0-py3-none-any.whl.metadata (1.6 kB)
Collecting click>=8.1.3 (from flask)
  Downloading click-8.3.3-py3-none-any.whl.metadata (2.6 kB)
Collecting itsdangerous>=2.2.0 (from flask)
  Downloading itsdangerous-2.2.0-py3-none-any.whl.metadata (1.9 kB)
Collecting Jinja2>=3.1.2 (from flask)
  Downloading Jinja2-3.1.6-py3-none-any.whl.metadata (2.9 kB)
Collecting MarkupSafe>=2.1.1 (from flask)
  Downloading MarkupSafe-3.0.3-cp313-cp313-win_amd64.whl.metadata (2.8 kB)
Collecting Werkzeug>=3.1.0 (from flask)
  Downloading Werkzeug-3.1.8-py3-none-any.whl.metadata (4.0 kB)
Collecting colorama (from click>=8.1.3->flask)
  Downloading colorama-0.4.6-py2.py3-none-any.whl.metadata (17 kB)
Downloading flask-3.1.3-py3-none-any.whl (103 kB)
Downloading blinker-1.9.0-py3-none-any.whl (8.5 kB)
Downloading click-8.3.3-py3-none-any.whl (110 kB)
Downloading itsdangerous-2.2.0-py3-none-any.whl (16 kB)
Downloading Jinja2-3.1.6-py3-none-any.whl (134 kB)
Downloading MarkupSafe-3.0.3-cp313-cp313-win_amd64.whl (15 kB)
Downloading Werkzeug-3.1.8-py3-none-any.whl (226 kB)
Downloading colorama-0.4.6-py2.py3-none-any.whl (25 kB)
Installing collected packages: markupsafe, itsdangerous, colorama, blinker, Werkzeug, Jinja2, click, flask
Successfully installed blinker-1.9.0 click-8.3.3 colorama-0.4.6 flask-3.1.3 itsdangerous-2.2.0 Jinja2-3.1.6 markupsafe-3.0.3 Werkzeug-3.1.8
```

```
PS C:\Users\User\OneDrive\Desktop\Task3> pip install bandit
Defaulting to user installation because normal site-packages is not writeable
Collecting bandit
  Downloading bandit-1.9.4-py3-none-any.whl.metadata (7.1 kB)
Collecting PyYAML>=5.3.1 (from bandit)
  Downloading PyYAML-6.0.3-cp313-cp313-win_amd64.whl.metadata (2.4 kB)
Collecting stevedore>=1.20.0 (from bandit)
  Downloading stevedore-5.7.0-py3-none-any.whl.metadata (2.4 kB)
Collecting rich (from bandit)
  Downloading rich-15.0.0-py3-none-any.whl.metadata (18 kB)
Requirement already satisfied: colorama>=0.3.9 in C:\Users\User\AppData\Local\Programs\Python\Python313\site-packages (from bandit) (0.4.6)
Collecting markdown-it-py>=2.2.0 (from rich->bandit)
  Downloading markdown_it_py-4.2.0-py3-none-any.whl.metadata (7.4 kB)
Collecting pygments<3.0.0,>=2.11.0 (from rich->bandit)
  Downloading pygments-2.20.0-py3-none-any.whl.metadata (2.5 kB)
Collecting mdurl>=0.1 (from markdown-it-py>=2.2.0->rich->bandit)
  Downloading mdurl-0.1.2-py3-none-any.whl.metadata (1.6 kB)
Downloading bandit-1.9.4-py3-none-any.whl (134 kB)
Collecting PyYAML-6.0.3-cp313-cp313-win_amd64.whl (154 kB)
Downloading stevedore-5.7.0-py3-none-any.whl (54 kB)
Downloading rich-15.0.0-py3-none-any.whl (310 kB)
Downloading pygments-2.20.0-py3-none-any.whl (1.2 MB)
1.2 MB 15.0 MB/s 0:00:00
Downloading markdown_it_py-4.2.0-py3-none-any.whl (91 kB)
Downloading mdurl-0.1.2-py3-none-any.whl (10.0 kB)
Installing collected packages: stevedore, PyYAML, pygments, mdurl, markdown-it-py, rich, bandit
Successfully installed bandit-1.9.4 markdown-it-py-4.2.0 mdurl-0.1.2 pygments-2.20.0 rich-15.0.0 stevedore-5.7.0
```

Screenshot 2 – Running Bandit Scan

```
PS C:\Users\User\OneDrive\Desktop\Task3> python -m bandit --version
main.py 1.9.4
python version = 3.13.7 (tags/v3.13.7:bcee1c3, Aug 14 2025, 14:15:11) [MSC v.1944 64 bit (AMD64)]
PS C:\Users\User\OneDrive\Desktop\Task3> python -m bandit -r .
[main] INFO profile include tests: None
[main] INFO profile exclude tests: None
[main] INFO cli include tests: None
[main] INFO cli exclude tests: None
[main] INFO running on Python 3.13.7
Run started:2025-05-16 07:47:25.910079+00:00

Test results:
>> Issue: [B608:hardcoded sql_expressions] Possible SQL injection vector through string-based query construction.
Severity: Medium confidence: Low
CWE: CWE-89 (https://cwe.mitre.org/data/definitions/89.html)
More Info: https://bandit.readthedocs.io/en/1.9.4/plugins/b608_hardcoded_sql_expressions.html
Location: .\app.py:14:12
13
14         query = "SELECT * FROM users WHERE username='" + username + "' AND password='" + password + "'"
15
-----
Code scanned:
Total lines of code: 16
Total lines skipped (#nosec): 0

Run metrics:
Total issues (by severity):
Undefined: 0
Low: 0
Medium: 1
High: 0
Total issues (by confidence):
Undefined: 0
Low: 1
Medium: 0
High: 0

Files skipped (0):
PS C:\Users\User\OneDrive\Desktop\Task3> 
```

Vulnerabilities Identified

Vulnerability 1: SQL Injection

Vulnerable Code

```
query = "SELECT * FROM users WHERE username='" + username + "' AND password='" + password + "'"
```

Description

The application directly concatenates user input into SQL queries. This allows attackers to manipulate database queries.

Risk Level: High

Impact

- Unauthorized login
- Database manipulation
- Data leakage

Example Attack

```
' OR '1'='1
```

This input may bypass authentication.

```
app.py 1 app.py...
1 from flask import Flask, request
2 import sqlite3
3
4 app = Flask(__name__)
5
6 @app.route('/login', methods=['POST'])
7 def login():
8     username = request.form['username']
9     password = request.form['password']
10
11     conn = sqlite3.connect('users.db')
12     cursor = conn.cursor()
13
14     query = "SELECT * FROM users WHERE username='" + username + "' AND password='" + password + "'"
15
16     cursor.execute(query)
17     result = cursor.fetchone()
18
19     if result:
20         return "Login Successful"
21     else:
22         return "Invalid Credentials"
```

Remediation

Secure Code

```
query = "SELECT * FROM users WHERE username=? AND password=?"
cursor.execute(query, (username, password))
```

Explanation

Parameterized queries prevent SQL injection by separating user input from SQL commands.

```
app.py 9+ app.py...
1 from flask import Flask, request
2 import sqlite3
3
4 app = Flask(__name__)
5
6 @app.route('/login', methods=['POST'])
7 def login():
8     username = request.form['username']
9     password = request.form['password']
10
11     conn = sqlite3.connect('users.db')
12     cursor = conn.cursor()
13
14     query = "SELECT * FROM users WHERE username=? AND password=?"
15     cursor.execute(query, (username, password))
16
17     cursor.execute(query)
18     result = cursor.fetchone()
19
20     if result:
21         return "Login Successful"
22     else:
23         return "Invalid Credentials"
24
25
```

Vulnerability 2: Hardcoded Credentials

Vulnerable Code

```
admin_password = "admin123"
```

Description

Sensitive credentials stored directly inside source code can be exposed if attackers access the application.

Risk Level: Medium

Impact

- Credential theft
- Unauthorized access

Remediation

Secure Approach

Use environment variables.

```
import os
admin_password = os.getenv("ADMIN_PASSWORD")
```

```
app.py 9+ app.py\...
1  from flask import Flask, request
2  import sqlite3
3
4  admin_password = "admin123"
5
6  app = Flask(__name__)
7
8  @app.route('/login', methods=['POST'])
9  def login():
10     username = request.form['username']
11     password = request.form['password']
```

Vulnerability 3: Plain Text Password Storage

Vulnerable Code

```
password = request.form['password']
```

Passwords are compared directly without hashing.

Risk Level: High

Impact

- Password theft
- Account compromise

Remediation

Secure Password Hashing

```
from werkzeug.security import generate_password_hash
from werkzeug.security import check_password_hash

hashed_password = generate_password_hash(password)
```

```
app.py 9+ app.py\...
1  from werkzeug.security import generate_password_hash
2  from werkzeug.security import check_password_hash
3
4  from flask import Flask, request
5  import sqlite3
6
7  admin_password = "admin123"
8
9  app = Flask(__name__)
10
11  @app.route('/login', methods=['POST'])
12  def login():
13     username = request.form['username']
14     password = request.form['password']
15
16     hashed_password = generate_password_hash(password)
17
18     conn = sqlite3.connect('users.db')
19     cursor = conn.cursor()
20
21     query = "SELECT * FROM users WHERE username=? AND password=?"
22     cursor.execute(query, (username, password))
23
24     cursor.execute(query)
25     result = cursor.fetchone()
26
27     if result:
28         return "Login Successful"
29     else:
30         return "Invalid Credentials"
```

Manual Code Review Findings

Issue	Severity	Recommendation
SQL Injection	High	Use parameterized queries
Hardcoded Password	Medium	Use environment variables
Plain Text Password	High	Use password hashing
Missing Input Validation	Medium	Validate user inputs
No Error Handling	Low	Implement exception handling

Secure Coding Best Practices

Recommended Practices

1. Use parameterized SQL queries
2. Encrypt or hash passwords
3. Avoid hardcoded secrets
4. Validate all user inputs
5. Use HTTPS for secure communication
6. Keep dependencies updated
7. Implement proper authentication
8. Use secure session management
9. Apply least privilege principle
10. Perform regular security testing

8. Improved Secure Login Example

```
from flask import Flask, request
from werkzeug.security import check_password_hash
import sqlite3

app = Flask(__name__)

@app.route('/login', methods=['POST'])
def login():
    username = request.form['username']
    password = request.form['password']

    conn = sqlite3.connect('users.db')
    cursor = conn.cursor()

    query = "SELECT password FROM users WHERE username=?"
    cursor.execute(query, (username,))

    result = cursor.fetchone()

    if result and check_password_hash(result[0], password):
        return "Login Successful"
    else:
        return "Invalid Credentials"
```

Screenshot – Final Secure Login Code

A screenshot of a code editor with a dark background. The code is written in Python and implements a secure login function. It includes imports for Flask, Werkzeug's security module, and sqlite3. The Flask app is initialized with __name__. A POST route for /login is defined. The login function extracts username and password from the request form, connects to a SQLite database named 'users.db', and executes a parameterized SQL query to find the password for the given username. It then uses Werkzeug's check_password_hash function to verify the password. If successful, it returns 'Login Successful', otherwise 'Invalid Credentials'. The code is wrapped in a main block that runs the app in debug mode.

```
1 from flask import Flask, request
2 from werkzeug.security import check_password_hash
3 import sqlite3
4
5 app = Flask(__name__)
6
7 @app.route('/login', methods=['POST'])
8 def login():
9     username = request.form['username']
10    password = request.form['password']
11
12    conn = sqlite3.connect('users.db')
13    cursor = conn.cursor()
14
15    # Secure parameterized query
16    query = "SELECT password FROM users WHERE username=?"
17    cursor.execute(query, (username,))
18
19    result = cursor.fetchone()
20
21    # Secure password verification
22    if result and check_password_hash(result[0], password):
23        return "Login Successful"
24    else:
25        return "Invalid Credentials"
26
27 if __name__ == '__main__':
28     app.run(debug=True)
29
```

(The final secure login implementation uses parameterized SQL queries and password hash verification to prevent SQL Injection and protect user credentials from exposure.)

Conclusion

The secure coding review identified multiple security vulnerabilities in the Flask login application. Major vulnerabilities included SQL Injection, hardcoded credentials, and insecure password handling.

By implementing parameterized queries, password hashing, environment variables, and proper validation techniques, the application security can be significantly improved.

The review demonstrates the importance of integrating security practices during software development.

Abstract

This project focuses on performing a secure coding review on a Python Flask-based login application. The objective of the review was to identify common security vulnerabilities such as SQL Injection, hardcoded credentials, and insecure password handling. Static analysis tools and manual inspection methods were used to analyze the source code. Appropriate remediation strategies and secure coding practices were also implemented to improve application security.